

Recursion

Recursion simply means: **a function calling itself** to solve a smaller part of the same problem.

Instead of solving everything at once, the problem is broken into smaller pieces until a **base condition** is reached (where recursion stops).

◆ Basic Structure of Recursion in C

```
void function() {  
    if (base_condition) {  
        return;  
    }  
    function(); // recursive call  
}
```

Whenever a function is called (recursive or not), the system uses a **stack** (called the *call stack*) to manage execution.

In recursion:

- Each recursive call gets its **own separate memory**
- That memory is stored in the **stack**

“Recursion is implemented using a stack where each function call creates an activation record containing parameters, local variables, and return address. These records are managed in a LIFO manner in the activation stack.”

◆ What is an Activation Record?

An **Activation Record (AR)** (also called a **Stack Frame**) is:

The memory block created in the stack for each function call

◆ Each Activation Record stores:

- Function parameters
- Local variables
- Return address (where to go back)
- Temporary data

◆ What is Activation Stack?

The **Activation Stack** is simply:

The stack that holds all activation records during program execution

So:

- Each function call → pushes an activation record
- Each return → pops it

◆ How Recursion Uses Stack (Example)

Let's take:

```
#include <stdio.h>
```

```
void fun(int n) {  
    if (n == 0)  
        return;  
    printf("%d ", n);  
    fun(n - 1);  
}
```

```
int main() {  
    fun(3);  
    return 0;  
}
```

◆ Execution Flow with Stack

Step 1: fun(3) is called

[fun(3)]

Step 2: fun(2) is called

[fun(3)]
[fun(2)]

Step 3: fun(1) is called

[fun(3)]
[fun(2)]
[fun(1)]

Step 4: fun(0) is called

[fun(3)]
[fun(2)]
[fun(1)]
[fun(0)]

Base condition reached

◆ Now stack starts popping (return phase)

Step 5:

fun(0) → returns

[fun(3)]
[fun(2)]
[fun(1)]

Step 6:

fun(1) → returns

[fun(3)]
[fun(2)]

Step 7:

fun(2) → returns

[fun(3)]

Step 8:

fun(3) → returns

[empty]

◆ Output

3 2 1

◆ Visualization (Push & Pop)

CALLS (Push)	RETURNS (Pop)
--------------	---------------

fun(3)	fun(0) returns
fun(2)	fun(1) returns
fun(1)	fun(2) returns
fun(0)	fun(3) returns

◆ Why Stack is Important in Recursion

Without stack:

- System cannot remember:
 - Where to return
 - Previous values of variables

That's why recursion works **only because of stack**

◆ Stack Overflow (Important Concept)

If recursion is too deep or missing base condition:

```
fun(n) {  
    fun(n+1); // no stop  
}
```

👉 Stack keeps growing → memory full → **Stack Overflow Error**

◆ Summary

- Recursion uses **call stack internally**
- Each call creates an **activation record**
- All records together form **activation stack**
- Execution:
 - Calls → **stack grows (push)**
 - Returns → **stack shrinks (pop)**

Types of Recursion:

◆ 1. Direct Recursion

- A function calls itself directly.
- **Example: Factorial**
- `#include <stdio.h>`

```
int fact(int n) {  
    if (n == 0)  
        return 1;  
    return n * fact(n - 1);  
}
```

```
int main() {  
    printf("%d", fact(4));  
    return 0;  
}
```

- ◆ **Output**

- 24

- ◆ **Recursion Tree**

- ```
fact(4)
 └─ 4 * fact(3)
 └─ 3 * fact(2)
 └─ 2 * fact(1)
 └─ 1 * fact(0)
 └─ 1
```

- ◆ **Explanation**

- `fact(0) = 1`  
`fact(1) = 1 * 1 = 1`  
`fact(2) = 2 * 1 = 2`  
`fact(3) = 3 * 2 = 6`  
`fact(4) = 4 * 6 = 24`
- 

## ◆ 2. Indirect Recursion

- 📌 One function calls another, which calls the first again.

- **Example**

- `#include <stdio.h>`

```
void funB(int n);
```

```
void funA(int n) {
 if (n <= 0) return;
 printf("%d ", n);
 funB(n - 1);
}
```

```
void funB(int n) {
 if (n <= 0) return;
 printf("%d ", n);
 funA(n / 2);
}
```

```
int main() {
 funA(5);
 return 0;
}
```

- ◆ **Output**

- 5 4 2 1

- ◆ **Tree Structure**

- `funA(5)`
    - └─ `print 5`
    - └─ `funB(4)`
      - └─ `print 4`
      - └─ `funA(2)`
        - └─ `print 2`
        - └─ `funB(1)`
          - └─ `print 1`
          - └─ `funA(0) → stop`
- 

## ◆ 3. Tail Recursion

- 📌 Recursive call is the **last operation** in the function.

- **Example**

- `#include <stdio.h>`

```
void printN(int n) {
 if (n == 0)
 return;
 printf("%d ", n);
 printN(n - 1);
}
```

```
int main() {
 printN(3);
 return 0;
}
```

-  **Output**

- 3 2 1

-  **Tree**

- ```
printN(3)
├── print 3
└── printN(2)
    ├── print 2
    └── printN(1)
        ├── print 1
        └── printN(0) → stop
```

- No work is left after recursive call → efficient (can be optimized by compiler)
-

4. Head Recursion

- Recursive call happens **before any processing**

- **Example**

- `#include <stdio.h>`

```
void printN(int n) {
    if (n == 0)
        return;
    printN(n - 1);
    printf("%d ", n);
}
```

```
int main() {
    printN(3);
    return 0;
}
```

-  **Output**

- 1 2 3

-  **Tree**

- ```
printN(3)
├── printN(2)
│ ├── printN(1)
│ │ └── printN(0)
```

```

 └─ print 1
 └─ print 2
 └─ print 3

```

- Work happens **while returning back (unwinding phase)**

### ◆ Important Insight

- In **tail recursion**, printing happens before recursive call  
In **head recursion**, printing happens during stack unwinding
- 

## ◆ 5. Tree Recursion

- A function makes **multiple recursive calls**
- **Example: Fibonacci**
- `#include <stdio.h>`

```

int fib(int n) {
 if (n <= 1)
 return n;
 return fib(n-1) + fib(n-2);
}

int main() {
 printf("%d", fib(4));
 return 0;
}

```

- ◆ **Output**

- 3

- ◆ **Tree Structure**

- `fib(4)`

```

 └─ fib(3)
 └─ fib(2)
 └─ fib(1) → 1
 └─ fib(0) → 0
 └─ fib(1) → 1
 └─ fib(2)
 └─ fib(1) → 1
 └─ fib(0) → 0

```

- Repeated calculations → inefficient
- 

## ◆ 6. Nested Recursion

The following nested recursion example is famously known as the McCarthy 91 Function, named after the computer scientist John McCarthy (one of the pioneers of Artificial Intelligence). It returns 91 for all inputs  $\leq 100$  and  $(n - 10)$  for inputs  $> 100$ .”

- Function calls itself inside its own parameter

- **Example**

- `#include <stdio.h>`

```
int fun(int n) {
 if (n > 100)
 return n - 10;
 return fun(fun(n + 11));
}

int main() {
 printf("%d", fun(95));
 return 0;
}
```

-  **Output**

- 91

-  **Tree (simplified)**

- fun(95)
  - fun(fun(106))
    - fun(96)
      - fun(fun(107))
        - fun(97)
          - ...
          - eventually returns 91

- This is complex but always converges to **91 for  $n \leq 100$**

- ---

# Summary Table

| Type     | Key Idea                  | Example Behavior |
|----------|---------------------------|------------------|
| Direct   | Calls itself              | Factorial        |
| Indirect | Functions call each other | funA ↔ funB      |
| Tail     | Call at end               | print reverse    |
| Head     | Call at beginning         | print forward    |
| Tree     | Multiple calls            | Fibonacci        |
| Nested   | Call inside parameter     | McCarthy-like    |

### ◆ Non-Tail Recursive Factorial (Common Version)

```
int fact(int n) {
 if (n == 0)
 return 1;
 return n * fact(n - 1);
}
```

---

### ◆ Why this is NOT tail recursion

Because **after the recursive call returns, some work is still pending**:

```
return n * fact(n - 1);
 ↑
```

The multiplication ( $n * \dots$ ) happens **after** the recursive call.

So:

- Function must **wait** for  $\text{fact}(n-1)$
- Then multiply
- Then return

That means it **cannot directly return the recursive call result**

---

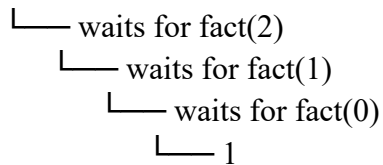
### ◆ Stack Behavior

For  $\text{fact}(4)$ :

```
fact(4)
→ 4 * fact(3)
→ 4 * (3 * fact(2))
→ 4 * (3 * (2 * fact(1)))
→ 4 * (3 * (2 * (1 * fact(0))))
→ 4 * 3 * 2 * 1 * 1
```

### ◆ Tree View

```
fact(4)
└── waits for fact(3)
```



All calls are **waiting** → **stack grows** → **then unwind happens**

---

### ◆ Tail Recursive Factorial (Modified Version)

We convert it using an **accumulator**

```
int fact_tail(int n, int result) {
 if (n == 0)
 return result;
 return fact_tail(n - 1, n * result);
}
```

Call it as:

```
fact_tail(4, 1);
```

---

### ◆ Why this is tail recursion

Now recursive call is the **last operation**

```
return fact_tail(n - 1, n * result);
```

- No work after the call
  - Result is already computed before calling
- 

### ◆ Flow

```
fact_tail(4,1)
→ fact_tail(3,4)
→ fact_tail(2,12)
→ fact_tail(1,24)
→ fact_tail(0,24)
→ 24
```

---

### ◆ Tree (Linear, No Waiting)

```
fact_tail(4,1)
└─ fact_tail(3,4)
 └─ fact_tail(2,12)
 └─ fact_tail(1,24)
 └─ fact_tail(0,24)
 └─ 24
```

No pending operations → no need to “come back”

---

### ◆ Key Difference

| Feature              | Normal Factorial | Tail Recursive Factorial |
|----------------------|------------------|--------------------------|
| Work after recursion | Yes              | No                       |
| Stack usage          | More             | Less (optimized)         |
| Execution style      | Expand + unwind  | Straight return          |

---

A recursion is **tail recursion** if:

“The recursive call is the last statement and its result is immediately returned.”